Architecture styles: what actually tells them apart

The names describe how something is deployed. What separates the styles is the answer to four questions, and a codebase can answer them in ways its name does not admit.

PREPARED BY
PRESENTER · TEAM

DATE
2026.08

AUDIENCE
ENGINEERING LEADS

Source — none. No figures appear in this deck; the argument is mechanical.

The label names a deployment, not a design decision

One name, two systems

“Microservices” covers a shop with one shared database and a shop with none. The word does not say which.

Two names, one system

A modular monolith and a service-based system can answer every question identically and differ only in where the process boundary falls.

The name is an outcome

Deployment shape is what the decisions produced. Reading it backwards tells you nothing about what was decided.

Every one of these is a naming problem, and none of them is a design problem.

Source — none. No figures appear in this deck; the argument is mechanical.

Four questions separate the styles you will meet

01

Deployment unit

What has to ship together for a change to take effect?

02

Data ownership

Who may write this table, and who may read it directly?

03

Call direction

Does the caller wait for the answer, or learn about it later?

04

Failure radius

When this part stops, what else stops with it?

Answer all four and the style is already chosen; answer none and the name is a guess.

Source — none. No figures appear in this deck; the argument is mechanical.

The same four answers place any codebase on the map

STYLE	DEPLOYMENT UNIT	DATA OWNERSHIP	CALL DIRECTION	FAILURE RADIUS
Monolith	One	Shared	In-process	Whole app
Modular monolith	One	Owned, in one DB	In-process	Whole app
Service-based	A few	Shared	Synchronous	A group
Microservices	Many	**Per service**	Synchronous	One service
Event-driven	Many	Per service	Asynchronous	One consumer

Only one column changes between service-based and microservices — and it is ownership, not count.

Source — none. No figures appear in this deck; the argument is mechanical.

When the label and the answers disagree, trust the answers

Distributed monolith

Many deployment units, one release train. Services ship together because they share a schema — the answers say monolith.

Microservices, one database

Per-service repositories over a shared schema. Ownership is the question that was never answered.

Modular monolith in name

One unit, one database, no owner per module. Calling it modular does not create the boundary.

Each of these ships the cost of the style it is named after, without the property that pays for it.

Source — none. No figures appear in this deck; the argument is mechanical.

What we need to answer before we name our style

01 **Which table has one writer**

Name it per module. Where two modules write the same table, the boundary is not there yet.

02 **What may we deploy alone**

If nothing may, the count of services is not the thing to change.

03 **What must survive a failure**

The parts that must stay up when another stops decide how far the boundary has to go.

The name follows the answers. It has never worked in the other direction.

Source — none. No figures appear in this deck; the argument is mechanical.